RB Discrete Flows report

ROHAN,NEAL BRIDGES

March 2026

Contents

1 Introduction & Historical Context

Flows are a continuation and a specific application of basic graph theory which can give us insight and new methods to solve problems of many different nature. For example, many problems of optimisation and finding maximum efficiency in a system are not only very easily solved by considering flows but are also the initial motivation and reasoning for developing the theory and definitions to come.
After World War II, many governments were allocating funds and resources to efficiency problems like supply routes, troop movement, and scheduling aircraft. This gave birth to operations research in the 1940s, and expressing these problems using graph theory and capacitated networks with the aim of finding the maximal flow. Major developments came in the 1950s with two mathematicians L.R.Ford and D.R.Fulkerson in their 1956 paper "Maximising flow through a Network", where they explain their Minimal Cut Theorem after being posed a question on railway traffic flows. This theorem helps us to find what the maximal flow of a system is and also helps make decisions on where improvements should be made to increase the maximal flow of the network.

2 Introductory Definitions

Constructing these capacitated networks begins by considering a simple graph with vertices and edges, like G = (V,E) for finite vertex and edge sets V and E respectively. Our start and end points of the system we call our source and sink, denoted **s** and **t** such that we are trying to maximise the flow from **s** to **t**. We introduce the capacities by saying that for every ordered pair $u, v \in V$, we have a capacity $c_{uv} \geq 0$ represented the maximum flow permitted through the edge $uv \in E$.
Note that if $uv \notin E$, $c_{uv} = 0$ since there is no joining edge joining u to v so no flow can go from u to v (at least directly). However, there may be $vu \in E$, in which case $c_{vu} > 0$. This is because we are working with directed networks here, and so the order of u and v is meaningful, unlike when E is an edge set of sets of pairs of vertices such as, E = {{u , v}...} where{u,v} = {v,u}. Consider the simple example of graph G = (V , E) for V = {**s** , a , b , c , **t**} E = {**s**a , ab , bc , c**t**} with capacities written below their respective edge.

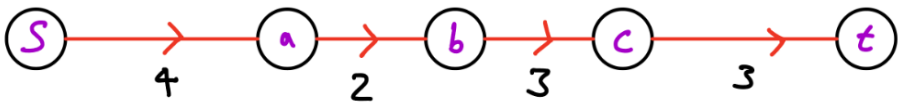

Figure 1: Example diagram illustrating the concept

One can clearly see that the maximum possible flow in this example is 2, since the directed edge ab is at its capacity and would not be able to hold any further

flow through it. This rule is one of the two main rules we must always obey in these networks.
(1) $f_{uv} \leq c_{uv} \quad \forall \quad u, v \in V$, where f_{uv} represents the flow between two vertices u and v. This simply means that the flow between two vertices is always limited by and cannot exceed its capacity.
(2) $\sum_{u \in V} f_{uv} - \sum_{u \in V} f_{vu} = 0$ for $v \notin \{s, t\}$, meaning that the total flow entering v is equal to the total flow leaving v. In other words, there is no leakage at any vertex v, provided $v \notin \{s, t\}$.
Note: If $v \in \{s, t\}$ this won't hold since the source can generate as much flow as is required and the sink can hold as much flow as is required.

3 Finding the Maximum flow

It is useful to keep in mind the likely real-world scenarios which we are simplifying here to be able to refer to our goals and stay focused on useful applications and results of our setup. In the examples of road/railway traffic, internet data routing and water supply, what is wanted is to find the maximum amount of flow which can be put through the system. These networks can get very large and finding this maximum flow is not always as simple as in Figure 1.

Example 1 Consider this capacitated network and attempt to find:

1) The maximum total flow of this network
2) An example achieving this maximum total flow.

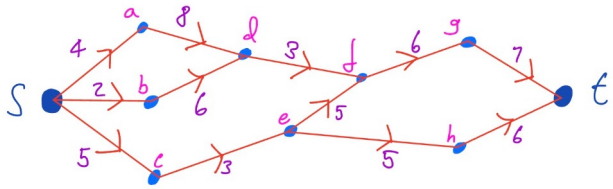

Figure 2: Slightly more complicated example

Since we have not developed any machinery to tackle this problem yet, we will have to use some form of trial and error on the flow through the network and reach a point where we can see no more flow can be added and so we have found the maximum flow and an example of it.
By considering the edges around the source and sink, we can immediately place an upper bound on what the max flow will be. Considering $\sum_{u \in V} c_{\mathbf{s}u}$ shows us the total capacity of all the edges leading out of our source, and therefore the total amount of flow that we could get leaving the source. In this case that gives us $4 + 2 + 5 = 11$. Similarly for the sink, $\sum_{u \in V} c_{u\mathbf{t}}$ gives us $7 + 6 = 13$ which is a maximum for the flow which could enter our sink. Therefore we can

place a bound of 11 on the maximum flow of our network, however, we can't be sure that it is the smallest bound, and therefore the maximum possible flow. So we will need to use a different method.

For example, one could try an iterative trial and improvement method where they slowly add 1 flow at a time until no more can be added while satisfying our rules. Putting a flow of 1 through the walk {**s** , a , d , f , g , **t**} and then the walk {**s** , c , e , h , **t**} gives a graph like this:

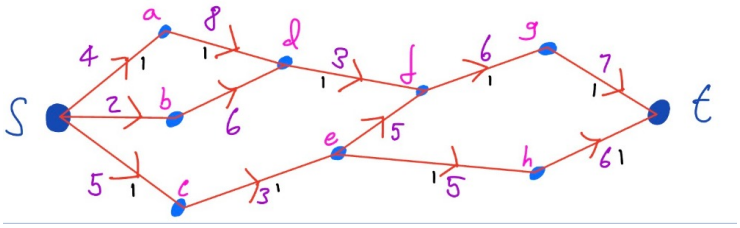

Figure 3: Iterative method

The black numbers around some edges represent the flow passing through them. One could next add a flow of 1 through the walk {**s** , b , d , f , g , **t**}. Now since f_{df}, f_{fg}, f_{gt} have now increased, we must check that they are still less than their respective capacities. Repeating this process until adding a flow of 1 to any possible walk from **s** to **t** would result in $f_{uv} > c_{uv}$ for some $u, v \in V$ will give us an example of the maximum flow going through the network. Then we can simply read off the amount of flow in the network by looking at the output of the source or the inflow into the sink and call it our maximum flow.
This could lead us to a graph looking like this:

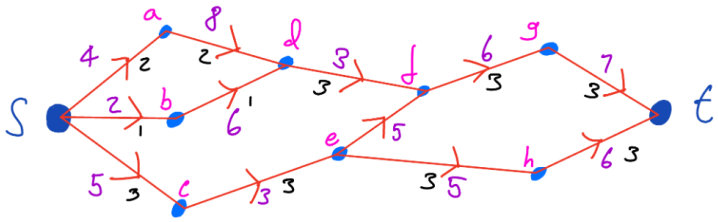

Figure 4: Max Flow example

Note: there is no unique way to achieve maximum flow in a network normally, but instead many different possibilities. However, after some investigating, we can see that in any way of getting a maximum flow in this example, $f_{df} = c_{df} = 3$ and $f_{ce} = c_{ce} = 3$.
These two edges act as a bottleneck for our network and are a limiting factor of the maximum flow we can achieve. Therefore, our maximum flow only occurs when these are maximised, and so our maximum flow is $c_{df} + c_{ce} = 6$.

This works since to get any flow from the source to the sink, it must go through either one of these edges, and so our maximum flow will be the maximum capacity of these edges. This is also the same logic used when trying to get a quick estimate on the max flow when considering the bounds imposed by the capacities around the source and sink. In reality, we have just found the minimum of these bounds with the edges {df , ce}. So, if we can generalise this concept of splitting the source and sink and seeing which edges are connecting them in different scenarios, and then finding the minimum of the sum of the capacities of those edges, we could gather a formula/method for finding the maximum possible flow in networks.

4 Cuts

Definition Let $S \subset V$ with $\mathbf{s} \in S$ and $\mathbf{t} \notin S$. The function $K : S \mapsto K(S)$ where $K(S) \subset E$ gives us the cut associated with S.

$$K(S) := \{uv \in E : u \in S, v \in V \setminus S\}$$

This now gives us a subset of E where all our edges start in S and end in $V \setminus S$. The **capacity** of K(S) is

$$k(S) := \sum_{e \in K(S)} c_e = \sum_{u \in S} \sum_{v \in V \setminus S} c_{uv}$$

Looking back to our previous example in Figure 2, the three cuts we had talked bout were surrounding the source and sink, and the one leading to finding our maximum flow. As cuts, they can be expressed as
1) $S_{source} = \{\mathbf{s}\}$ with $k(S_{source}) = 11$
2) $S_{sink} = V \setminus \{\mathbf{t}\}$ with $k(S_{sink}) = 11$
3) $S_{min} = \{\mathbf{s}, a, b, c, d\}$ with $k(S_{min}) = 6$
Recall: all that was special about S_{min} was that it lead to finding the limiting point of our network since $k(S_{min})$ was a minimum for this S_{min}. Therefore, taking the minimum of k(S) over all possible sets S, should give us our maximum flow.

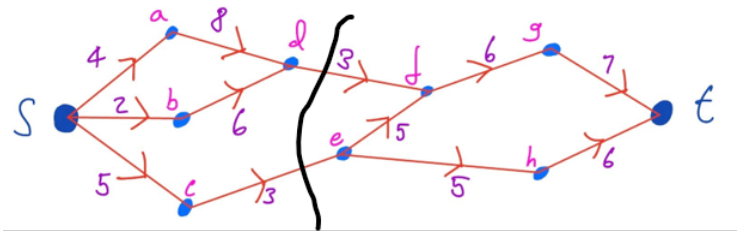

Figure 5: Visual representation of the cut

This is precisely why the term "cut" is used, we can visualise this as cutting the

graph into to parts, one with **s**, one with **t**, and to get from **s** to **t** any flow must pass through the edges included in K(S). (Figure 5 shows us the associated cut where $S = S_{min}$)

5 Max-flow Min-cut Theorem

This theorem is the one which Ford and Fulkerson were looking at and offer a proof of (different to what you're going to see here) in their paper aforementioned.

Theorem:

$$\max_{\boldsymbol{F}} \boldsymbol{\Phi_{tot}(F)} = \min_{\boldsymbol{S}} \boldsymbol{k(S)}$$

We have already looked at what the right hand side of this equation means and its significance. The left hand side is taking the maximum over all flows F, of $\Phi_{\text{tot}}(F)$. $\Phi_{\text{tot}}(F)$ simply represents the total flow going through the network. Expressing this as a summation, we could use $\sum_{v \in V} f_{sv} - \sum_{u \in V} f_{us}$ or $\sum_{v \in V} f_{vt} - \sum_{u \in V} f_{tu}$ which represent the total flow generated by the source and total flow received by the sink. We know that these will be equal to each other and to the total flow in our network due to rule **(2)**, having no leakage through any edge or vertex which isn't **s** or **t**. Therefore, all that is generated by the source must get received by the sink.

Proof: Unfortunately, our previous work on understanding why there is intuition as to why these two would be equal is not adequate for a proof. This proof will take lots of sum manipulations and setting up new functions. We will also split the proof into two parts and achieve our desired result via trichotomy.

$$\max_{\boldsymbol{F}} \boldsymbol{\Phi_{tot}(F)} \leq \min_{\boldsymbol{S}} \boldsymbol{K(S)}.$$

First, we define new functions $\phi_{\text{in}}, \phi_{\text{out}}$ for $W \subset V$

$$\phi_{\text{in}}(W) := \sum_{u \in V \setminus W} \sum_{v \in W} f_{uv} \ , \ \phi_{\text{out}}(W) := \sum_{u \in W} \sum_{v \in V \setminus W} f_{uv}$$

Lemma 1. $\phi_{\text{in}}(W \cup \{y\}) = \phi_{\text{in}}(W) + \sum_{u \in V \setminus W} f_{uy} - \sum_{v \in W} f_{yv}$ for $y \in V \setminus W$

PROOF:

$$\begin{aligned}
\phi_{\text{in}}(W \cup \{y\}) &= \sum_{u \in V \setminus (W \cup \{y\})} \sum_{v \in W \cup \{y\}} f_{uv} \\
&= \sum_{u \in V \setminus (W \cup \{y\})} \left(\sum_{v \in W} f_{uv} + f_{uy} \right) \\
&= \sum_{u \in V \setminus (W \cup \{y\})} \sum_{v \in W} f_{uv} + \sum_{u \in V \setminus (W \cup \{y\})} f_{uy} \\
&= \sum_{u \in V \setminus W} \sum_{v \in W} f_{uv} - \sum_{v \in W} f_{yv} + \sum_{u \in V \setminus W} f_{uy} - f_{yy} \\
&= \phi_{\text{in}}(W) + \sum_{u \in V \setminus W} f_{uy} - \sum_{v \in W} f_{yv}
\end{aligned}$$

Lemma 2. $\phi_{\text{out}}(W \cup \{y\}) = \phi_{\text{out}}(W) - \sum_{u \in W} f_{uy} + \sum_{v \in V \setminus W} f_{yv}$ for $y \in V \setminus W$

PROOF:

$$\begin{aligned}
\phi_{\text{out}}(W \cup \{y\}) &= \sum_{u \in W \cup \{y\}} \sum_{v \in V \setminus (W \cup \{y\})} f_{uv} \\
&= \sum_{u \in W \cup \{y\}} \left(\sum_{v \in V \setminus W} f_{uv} - f_{uy} \right) \\
&= \sum_{u \in W} \left(\sum_{v \in V \setminus W} f_{uv} - f_{uy} \right) + \left(\sum_{v \in V \setminus W} f_{yv} - f_{yy} \right) \\
&= \phi_{\text{out}}(W) - \sum_{u \in W} f_{uy} + \sum_{v \in V \setminus W} f_{yv}
\end{aligned}$$

In proving these first 2 lemmas, all we have done is split the sums into a sum over W or $W \setminus V$, then adding or subtracting the term in the sum when our variable = y.
Then also noting that $f_{yy} = 0$ and vanishes since there are no self-loops in these simple graphs we are considering and so no flow from itself to itself.

Now let $\Delta(W) := \phi_{\text{out}}(W) - \phi_{\text{in}}(W)$ for $W \subset V$

Lemma 3. $\Delta(v) = 0$ for $v \in V \setminus \{s, t\}$

PROOF:

$$\begin{aligned}
\Delta(v) &= \phi_{\text{out}}(v) - \phi_{\text{in}}(v) \\
&= \sum_{u \in V} \sum_{k \in V \setminus \{v\}} f_{uk} - \sum_{j \in V \setminus \{v\}} \sum_{m \in \{v\}} f_{jm} \\
&= \sum_{k \in V \setminus \{v\}} \sum_{u \in \{v\}} f_{uk} - \sum_{j \in V \setminus \{v\}} \sum_{m \in \{v\}} f_{jm} \\
&= \sum_{k \in V \setminus \{v\}} (f_{vk} - f_{kv}) \\
&= 0 \qquad \text{since there is no leakage in our system - rule } \textbf{(2)}
\end{aligned}$$

Lemma 4. Let $W \subset V$, $y \in V \setminus W$. Then
$\Delta(W \cup \{y\}) = \Delta(W) + \sum_{v \in V \setminus W} f_{yv} - \sum_{u \in W} f_{uy}$

PROOF:

$$\begin{aligned}
\Delta(W \cup \{y\}) &= \phi_{\text{out}}(W \cup \{y\}) - \phi_{\text{in}}(W \cup \{y\}) \\
&= \left[\phi_{\text{out}}(W) + \sum_{v \in V \setminus W} f_{yv} - \sum_{u \in W} f_{uy} \right] - \left[\phi_{\text{in}}(W) + \sum_{u \in V \setminus W} f_{uy} - \sum_{v \in W} f_{yv} \right] \\
&= [\phi_{\text{out}}(W) - \phi_{\text{in}}(W)] + \left[\sum_{v \in V \setminus W} f_{yv} + \sum_{v \in W} f_{yv} \right] - \left[\sum_{u \in W} f_{uy} + \sum_{u \in V \setminus W} f_{uy} \right] \\
&= \Delta(W) + \sum_{v \in V} f_{yv} - \sum_{u \in V} f_{uy}
\end{aligned}$$

Here we have used our result from Lemmas 1 & 2 to expand our ϕ_{in} and ϕ_{out} expressions. Then we have also used $V \setminus W$ and W being a partition of V to put the two sums into one sum over all of the elements in V

Proof by induction. We prove that $\Delta(W) = 0$ if $s, t \notin W$.

Base case. $|W| = 1$, so $W = \{v\}$ for some $v \in V$. We know that $\Delta(W) = \Delta(\{v\}) = 0$. by Lemma 3. Therefore, the result holds in the base case.

Inductive hypothesis. Assume that for some $W \subset V$ with $|W| = n$ and $s, t \notin W$, we have

$$\Delta(W) = 0.$$

Inductive step. Consider $W' = W \cup \{y\}$, where $y \in V \setminus W$ and $y \neq s, t$, such that $|W'| = n + 1$ Then

$$\Delta(W') = \Delta(W \cup \{y\}) = \Delta(W) + \sum_{v \in V} f_{yv} - \sum_{u \in V} f_{uy}.$$

By the inductive hypothesis, $\Delta(W) = 0$. Also,

$$\sum_{v \in V} f_{yv} - \sum_{u \in V} f_{uy} = 0,$$

due to rule **(2)** again, there is no leakage in our network at a point y where $y \neq \mathbf{s}, \mathbf{t}$

Hence,

$$\Delta(W') = 0.$$

Conclusion. By induction, $\Delta(W) = 0$ for all $W \subset V$ with $\mathbf{s}, \mathbf{t} \notin W$

Now with the help of our Lemmas, we can look back at our "cutting sets" $S \subset V$with $\mathbf{s} \in S, \mathbf{t} \notin S$

If $\mathbf{s} \in S$, $\mathbf{t} \notin S$

$$\Delta(S) = \Delta(S' \cup \{s\}), \quad \text{Where } S' = S \setminus \{s\}$$

$$\Delta(S) = \Delta(S') + \sum_{v \in V} f_{sv} - \sum_{u \in V} f_{us} \quad \text{by Lemma 4}$$

$$\Delta(S') = 0 \quad \text{by induction result, since } \mathbf{s} \notin S',\ \mathbf{t} \notin S'.$$

$$\text{So} \quad \Delta(S) = \sum_{v \in V} f_{sv} - \sum_{u \in V} f_{us}$$

$$\Delta(S) = \Phi_{\text{tot}}(F) \quad \text{as defined}$$

Recall: We defined $\Phi_{\text{tot}}(F)$ as the total flow generated by our source, or received by our sink, which can be expressed as the sums above.

$$So \quad \Phi_{\text{tot}}(F) = \phi_{\text{out}}(S) - \phi_{\text{in}}(S)$$

$$\Phi_{\text{tot}}(F) = \sum_{u \in S} \sum_{v \in V \setminus S} f_{uv} - \sum_{u \in V \setminus S} \sum_{v \in S} f_{uv}$$

$$\Phi_{\text{tot}}(F) \leq \sum_{u \in S} \sum_{v \in V \setminus S} f_{uv} \qquad \text{since} \quad f_{uv} \geq 0 \quad \forall \quad u, v \in V$$

$$\Phi_{\text{tot}}(F) \leq \sum_{u \in S} \sum_{v \in V \setminus S} c_{uv} = k(S)$$

So for all S as defined above, $\Phi_{\text{tot}}(F) \leq k(S)$

This implies:

$$\max_{F} \Phi_{\text{tot}}(F) \leq \min_{S} k(S)$$

We are going to omit the proof that $\max_F \Phi_{\text{tot}}(F) \geq \min_S K(S)$. and concentrate instead on the applications and insights we can gather from the inequality in which we have proved. However, gaining this lower bound on the maximum flow can be achieved by considering walks from our source to sink, and seeing if we can increase the flow through that specific walk, obtaining a maximum flow that way. This is in very similar fashion to that which we employed in our iterative approach to finding a maximum flow in figure 2.

6 Applications & Uses

Now we are going to see just how this inequality can be used in real optimisation and planning problems.

6.1 Car Motorway Traffic

Imagine a multitude of cars are driving from London to Durham in late September. Local councils however are needing to perform roadworks on some of the motorway lanes and so would ideally like to clodse off some lanes of traffic. What is the most optimal solution, still ensuring everybody can get from London to Durham without a majorly disrupted journey?
Constructing a simplified diagram of the motarway system connecting London to Durham can help us gain a clearer view on what our best course of action is.

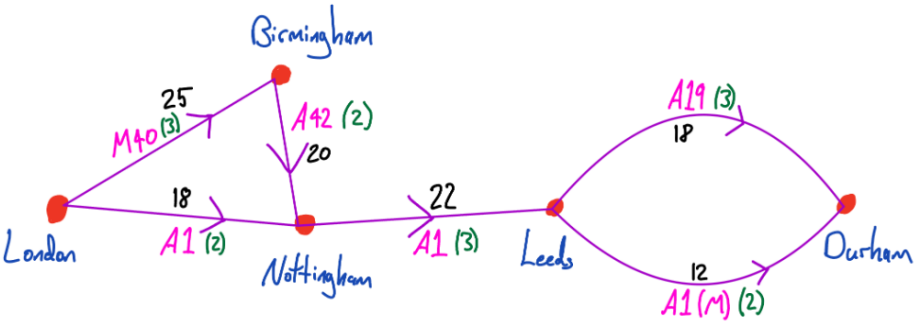

Figure 6: London to Durham

Here, the black numbers represent the number of hundreds of cars per hour capacity of the road.
And the green number is the amount of lanes of that road.

Note that the numbers used for lanes and capacities are purely made up without any research, and are just a rough estimate used for this illustration.

Using our proven inequality, we can take a set of S = {London , Birmingham , Nottingham} which has a cut capacity of k(S) = 22 hundred cars per hour. And so, we know that the maximum possible flow through our system will be 2200 cars/hour.

Therefore, we can advise lane closures of anything that would not amount to creating a possible smaller cut capacity in our network.
Assuming a linear relationship between amount of lanes and capacity of road, we can see that closing one lane in both the A1 and A42 would leave us with the set W = {London , Birmingham} giving a cut capacity of k(W) = 9 + 10 = 19 < 22, and so would result in increased total disruptions to journeys.
After doing some looking and thinking, we can see that its possible to be able to close 2 lanes on the M40, 1 on the A42, and 1 on the A19 with still being able to have a flow of 22 hundred cars/hour.

Obviously there are some major caveats to this example, firstly concerning imperfect information amongst drivers, potential accidents causing ruined journeys if only 1 lane is active on a given motorway. However, it shows the uses of this theorem and these diagrams and their potential in solving extremely large optimisation problems.

6.2 Data Trafficking

In this example of data, it is much easier to see how this can be used at such a large scale, especially when considering server rooms in office buildings, of which there must be hundreds in London for example. Therefore, we can see how it would be very costly to have inefficient data movement.
When translating our example into this context, we call our vertices nodes (computers, servers, routers), the edges are cables (with the capacity being their maximum bandwidth) and the flow is the actual data being sent through the cables, from node to node. It is easy to imagine the complexity of these systems, as even within each node these networks are occurring on smaller scales. Consider a computer for example, lots of data is constantly running through different nodes inside the computer, receiving, interpreting and transferring new data constantly. Consequently these networks can get extremely complex and so may require computing power and code to help analyse the graphs we can make the networks into. Through sheer code and trial and improvement, we can get the computers to identify the minimum cut in these extremely large systems, which we can then use to identify where to allocate improvements to. Since this minimum cut places a bound on our maximum flow, we can improve this part of the network to gain a larger maximum output.

7 References & declaration

Ford, L.R. and Fulkerson, D.R. (1956). Maximal Flow Through a Network. Canadian Journal of Mathematics, 8, pp.399–404. doi:https://doi.org/10.4153/cjm-1956-045-5.

Wikipedia Contributors (2019). Maximum flow problem. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Maximum_flow_problem.

Declaration.

I have read and understood these guidelines. I also understand that any plagiarism could have serious consequences for my mark, and for my academic career.

Name: ROHAN BRIDGES

Signed: ..... Date 20/05/2020